

DBMS LAB PROJECT · 2026

# LuxBid

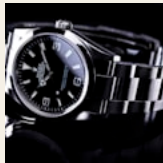
*A Luxury Watch Auction Platform*

— POSTGRESQL · NODE.JS · REACT —

# Project Overview

*"The global luxury watch market exceeds \$52B — yet auction platforms lack the transactional integrity the trade demands."*

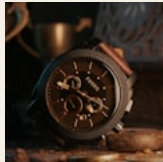
LuxBid addresses this with a backend built entirely around PostgreSQL's most powerful features.



FEATURED LOT · 247

A. Lange & Söhne · Lange 1 "Daylight"

Est. \$42,000 – \$56,000



FEATURED LOT · 311

Rolex · Daytona Ref. 6263 "Paul Newman"

Est. \$85,000 – \$110,000



FEATURED LOT · 189

Rexhep Rexhepi · Akria Chronomètre Contemporain

Est. \$120,000+

# Platform Features



01

## User Registration & Auth

Buyer & seller roles, JWT sessions, hashed credentials stored in PostgreSQL.

02

## Watch Listings & Catalog

Hierarchical categories (brand → line → variant) traversed via recursive CTEs.

03

## Live Bidding Engine

Atomic bid placement via stored procedures; concurrent bids resolved by ACID transactions.

04

## Auction Timer & Auto-close

A scheduled procedure with a cursor iterates all expired auctions and finalises transactions.

05

## Transaction History

Full audit trail via a dedicated audit\_log table, queryable with window functions for analytics.

06

## Smart Notifications

Outbid & auction-close alerts fired automatically by database-level AFTER INSERT triggers.

# Architecture



# DBMS Features in LuxBid

|    | PLATFORM FEATURE        | DATABASE TECHNIQUE                  | SQL CONSTRUCT       |
|----|-------------------------|-------------------------------------|---------------------|
| 01 | Outbid Notification     | Event-driven alert on INSERT        | TRIGGER             |
| 02 | Auction Auto-close      | Scheduled batch finalisation        | TRIGGER + PROCEDURE |
| 03 | Place Bid (Validation)  | Transactional business logic        | STORED PROCEDURE    |
| 04 | Auction Leaderboard     | Leading bid per auction             | CTE + ROW_NUMBER()  |
| 05 | Top Bidder Analytics    | Ordered aggregation ranking         | WINDOW FUNCTIONS    |
| 06 | Expire Auctions (Batch) | Row-by-row procedural iteration     | CURSOR              |
| 07 | Watch Category Tree     | Hierarchical traversal              | RECURSIVE CTE       |
| 08 | Concurrent Bid Safety   | Conflict-safe all-or-nothing commit | ACID TRANSACTION    |
| 09 | Fast Bid Query Lookup   | Sorted index scan                   | COMPOSITE INDEX     |
| 10 | Live Auction Report     | Pre-computed result set             | MATERIALIZED VIEW   |

# Triggers

Functions that execute automatically when a specified database event occurs — no application code required on the notification path.

- FOR EACH ROW gives access to the NEW and OLD record — the trigger sees exactly what changed
- Can INSERT, UPDATE, or RAISE EXCEPTION to abort the entire operation with a rollback
- Multiple triggers can chain — bid INSERT fires notification trigger, which fires audit trigger

TRG\_OUTBID · AFTER INSERT ON BIDS

## Outbid Notification

Finds the previous highest bidder from the bids table, then writes an alert to the notifications table. Fully decoupled from the Node.js API — no application code needed.

TRG\_CLOSE\_AUCTION · AFTER UPDATE ON AUCTIONS

## Auction Auto-close

When an auction's status flips to 'closed', the trigger automatically creates the transaction record linking the winning bid to buyer and seller.

TRG\_AUDIT\_BID · AFTER INSERT ON BIDS

## Bid Audit Log

Appends an immutable, timestamped entry to audit\_log on every bid placed — creating a tamper-evident record of all platform activity.

# Stored Procedures

Named PL/pgSQL routines that run entirely inside PostgreSQL — encapsulating validation, business logic, and transactional control in one compiled unit.

- DECLARE block scopes local variables — v\_max, v\_status — for bid validation before any write occurs
- RAISE EXCEPTION propagates typed errors back to Node.js, triggering automatic rollback of the whole transaction
- COALESCE on MAX(bid amount) falls back gracefully to starting\_price when no bids exist yet

PLACE\_BID(USER, AUCTION, AMOUNT)

## Place Bid

Validates the new amount exceeds the current max, then inserts atomically. Raises a typed exception if the bid is too low or the auction is inactive.

CLOSE\_EXPIRED\_AUCTIONS()

## Close Auctions

Cursor-driven loop over all ended auctions. Closes each one and creates the winner's transaction record before moving to the next row.

AWARD\_WINNER(AUCTION\_ID)

## Award Winner

Creates the transaction record linking buyer, seller, and watch. Queues congratulatory and settlement notifications for both parties.

GET\_LEADERBOARD(AUCTION\_ID)

## Leaderboard

Returns ranked bid history for a given auction as a result set — joining bids, users, and the watch record in one efficient call.

# CTEs & Window Functions

---

## COMMON TABLE EXPRESSIONS

### WITH name AS (...)

Named subqueries that define a reusable intermediate result set scoped to one query — making multi-step logic readable and maintainable.

---

- **ranked\_bids** — base table for the leaderboard: selects and ranks all bids in one named step
- **WITH RECURSIVE cat\_tree** — traverses the watch category hierarchy to any depth
- **bid\_summary** — per-user analytics computed once, referenced multiple times in the outer query

## WINDOW FUNCTIONS

### OVER (PARTITION BY...)

Compute aggregate values across a set of related rows without collapsing them — unlike GROUP BY, every original row is preserved in the output.

---

- **ROW\_NUMBER() PARTITION BY auction\_id** — identifies the single leading bid per auction
- **RANK() ORDER BY amount DESC** — assigns a global position to every bid for the platform leaderboard
- **SUM(amount) OVER (seller\_id)** — running revenue total per seller without a separate GROUP BY pass

# Cursors

A named pointer that traverses a query result one row at a time inside a stored procedure — essential for safe batch operations on large datasets.

- FOR UPDATE locks each row as it is fetched — preventing concurrent modifications mid-loop
- WHERE CURRENT OF updates the locked row without a second index scan — efficient and safe
- EXIT WHEN NOT FOUND terminates the loop cleanly once all rows are processed

## CLOSE\_EXPIRED\_AUCTIONS() – CURSOR LIFECYCLE

- 1 DECLARE**  
Define the cursor: SELECT all auctions WHERE end\_time < NOW() AND status = 'active' FOR UPDATE
- 2 OPEN**  
PostgreSQL executes the query and positions the cursor before the first row
- 3 FETCH INTO rec**  
Retrieves one auction at a time into a RECORD variable for processing
- 4 Process**  
UPDATE status → 'closed' WHERE CURRENT OF, then INSERT the winning bid as a transactions record
- 5 CLOSE**  
Loop exits on NOT FOUND; cursor is closed and all row locks are released

# ACID Transactions

---

## A

### Atomicity

Every bid INSERT, notification write, and audit\_log entry execute as a single unit. If any step fails, all changes are rolled back — the database never lands in a partial state.

## C

### Consistency

Foreign key constraints, CHECK clauses on bid amounts, and trigger validations all enforce domain rules within the transaction boundary. An invalid bid simply cannot persist.

## I

### Isolation

READ COMMITTED prevents dirty reads. FOR UPDATE in the cursor loop serialises concurrent bids on the same auction — two users cannot simultaneously claim the winning position.

## D

### Durability

Once COMMIT returns, PostgreSQL's Write-Ahead Log guarantees the bid record survives a power failure, server restart, or crash. Every trade is permanently on the books.

# Indexes, Views & Recursive CTEs

COMPOSITE & PARTIAL INDEXES

## Fast Lookups

A composite index on bids(auction\_id, amount DESC) makes "find the top bid for auction #7" an instant sorted scan — avoiding a full table read on every page load.

A partial index on auctions(end\_time) WHERE status = 'active' indexes only live auctions — keeping the index small, tight, and fast for the queries that matter most.

MATERIALIZED VIEW

## Pre-computed Results

mv\_leaderboard pre-joins watches, auctions, and bids and stores the result physically on disk. Read-heavy requests — live auction pages refreshing every few seconds — hit the view directly without re-joining on every call.

REFRESH MATERIALIZED VIEW is called by the close\_auction() procedure after each auction closes, keeping the data current.

RECURSIVE CTE

## Hierarchy Traversal

WITH RECURSIVE cat\_tree starts with root categories (parent\_id IS NULL), then repeatedly joins children onto their parents — traversing brand → collection → model → variant to any depth in a single query.

Powers the "Browse by brand" catalog filter and generates nested breadcrumb navigation across the entire watch hierarchy.

# Thank You

*Questions?*

## 10 POSTGRESQL TECHNIQUES

- TRIGGER
- STORED PROCEDURE
- CTE
- WINDOW FUNCTIONS
- CURSOR
- RECURSIVE CTE
- ACID TRANSACTION
- COMPOSITE INDEX
- MATERIALIZED VIEW
- PL/pgSQL

